

What Is the Memory System?

Claude Code has a built-in persistent memory system. It stores key facts about you, your project, and your preferences in small markdown files on your computer. These files are read automatically at the start of every new chat session.

This is how Claude "remembers" who you are and what you were working on - even after closing VS Code and coming back the next day.

Where the Files Live (Your Project)

```
C:\Users\Administrator\.claude\projects\  
c--Users-Administrator-Downloads-Python-Training\  
memory\  
  MEMORY.md          <- the index (always loaded first)  
  user_profile.md     <- who you are  
  project_context.md  <- what the project is about  
  feedback_style.md   <- how you like Claude to behave
```

The folder name "c--Users-Administrator-Downloads-Python-Training" is just Claude Code's way of turning your project folder path into a safe folder name (replacing \ and : with -).

MEMORY.md - The Master Index

MEMORY.md is the only file Claude reads on EVERY session start. It is a short index (max 200 lines) that lists all memory files and a one-line summary of each.

Your current MEMORY.md looks like this:

```
# Memory Index  
  
- [User Profile](user_profile.md) - Mohammad Kamran, Python learner,  
  prefers simple analogies and layman explanations  
  
- [Project Context](project_context.md) - Python Training workshop;  
  pin_extractor.py built so far; concepts covered  
  
- [Feedback & Style](feedback_style.md) - Use 7-year-old analogies,  
  show before/after code, use user's own files as examples
```

The 200-line limit means: key facts only, not full transcripts. Think sticky notes, not a diary.

Memory File Format

Every individual memory file has two parts: a frontmatter header and the actual content.

```
---  
name: user-profile  
description: Who the user is and how they like to learn  
metadata:  
  type: user  
---
```

Level 2 - Memory System (MEMORY.md) - Python Training Notes

Mohammad is learning Python through a workshop format.
Prefers simple explanations with real examples.
Uses VS Code on Windows 11.

The section between --- is the frontmatter (machine-readable metadata).

Everything below it is the memory content Claude actually uses.

The 4 Types of Memory

Type	What It Stores	Example
user	Who you are, skill level, preferences	Mohammad, Python learner, likes analogies
feedback	What Claude should/should not do	Use 7-year-old explanations; no long comments
project	Goals, decisions, what has been built	pin_extractor.py; concepts covered so far
reference	Where to find things - files, tools, links	pin_extractor.py is in Python Training folder

The Exact Flow - What Happens on Every New Chat

```
You open VS Code and start a new Claude Code chat
|
Claude reads MEMORY.md first (always, automatically)
|
MEMORY.md lists 3 files with one-line summaries
|
Claude loads the relevant ones into its context
|
Claude now knows:
- You are Mohammad, learning Python
- You are working on pin_extractor.py
- You like simple analogies
- What Python topics you have already covered
|
You type: "continue where we left off"
|
Claude already knows the full picture - no re-explaining needed
```

What You Can Do With This System

Tell Claude to remember something:

```
"Remember that I don't like long code comments"
-> Claude writes a new memory file
-> Adds a line to MEMORY.md
```

Tell Claude to forget something:

```
"Forget that preference about X"
-> Claude finds and removes that memory file
-> Removes the line from MEMORY.md
```

Tell Claude to update memory:

```
"We have now also covered decorators and list comprehensions"
-> Claude updates project_context.md with new info
```

Ask Claude what it remembers:

```
"What do you remember about me?"
-> Claude reads and summarises all memory files for you
```

Your 3 Memory Files (Created This Session)

1. user_profile.md

```
Name: Mohammad Kamran
Type: user

- Learning Python through a structured workshop format
```

- Prefers simple, layman explanations
- Asks conceptual "why" questions alongside practical coding
- Uses VS Code on Windows 11 with Claude Code extension
- Interested in how tools work under the hood

2. project_context.md

Type: project

Main file: pin_extractor.py

Built so far:

- pin_extractor(poems) takes a list of poems
- Returns a list of secret codes
- Each code digit = length of the word at index = line_index

Concepts covered:

split(), enumerate(), isinstance(), not, range(),
insert(), clear(), strip(), triple-quoted strings

3. feedback_style.md

Type: feedback

- Use simple analogies and "7 year old" style explanations
- Show wrong code AND corrected version side by side
- Keep responses concise unless deep explanation is asked for
- Use the user's own files as examples wherever possible

The One Limitation

MEMORY.md is capped at 200 lines. Once you hit that, older entries get dropped.

This is why memory is for KEY FACTS only - not full conversation transcripts.

Think of it like sticky notes on your desk:

Good: "Mohammad prefers analogies"

Bad: Copy-pasting the entire conversation history

Memory vs Compaction - What Is the Difference?

Feature	Memory Files	Compaction
What it is	You write key facts to .md files	Claude summarises old chat messages
When it runs	You tell Claude to remember	Automatically when context fills up
Survives new chat?	YES - persists forever	NO - only within current session
What is stored	Curated facts you chose to keep	Summary of conversation so far
You control it?	YES - add, edit, delete anytime	NO - happens automatically